



Universidad Nacional de San Luis

Facultad de Ciencias Físico Matemáticas y
Naturales

Departamento de Informática

Práctico Final Integrador
Sistemas Distribuidos y Paralelos

Generación de Cartoons a partir de Imágenes

Juan Cruz Paez
María Lujan Flores Medina

Profesores:
Fabiana Piccoli
Cesar Ochoa

Ingeniería en Informática

2026

Índice general

1. Introducción	1
2. Decisiones de Diseño	3
2.1. Paradigmas y estrategia general	3
2.2. Algoritmo secuencial	3
2.3. Algoritmo con memoria compartida (OpenMP)	3
2.4. Algoritmo con memoria distribuida (MPI)	4
2.5. Algoritmo híbrido (MPI + OpenMP)	4

Capítulo 1

Introducción

El procesamiento digital de imágenes constituye una de las áreas más demandantes en términos computacionales dentro de la informática moderna. Si bien las bases matemáticas de muchas de sus técnicas datan de siglos atrás, su aplicación práctica a escala masiva solo fue posible con el avance tecnológico del hardware de las últimas décadas, que permitió realizar en tiempo razonable la enorme cantidad de operaciones aritméticas involucradas.

Una imagen digital puede representarse como un arreglo bidimensional de píxeles. En el modelo de color **RGB** (Red, Green, Blue) utilizado en este trabajo, cada píxel contiene tres valores enteros en el rango $[0, 255]$, uno por canal de color. Esta representación permite codificar hasta 16,8 millones de tonalidades distintas, superando la capacidad discriminatoria del ojo humano. Los casos extremos son el negro $(0, 0, 0)$ y el blanco $(255, 255, 255)$.

El objetivo del presente trabajo es la **generación automática de imágenes tipo *cartoon*** a partir de una imagen original en formato RGB. Esta transformación busca simplificar formas, resaltar contornos y reducir variaciones tonales, logrando una apariencia similar a la de una ilustración o dibujo animado. Para ello se aplican dos procesos independientes sobre la imagen original:

- **Posterización:** reduce el rango total de niveles de color a un conjunto más pequeño de valores discretos. Cada píxel pasa a adoptar el valor del nivel más cercano dentro de ese rango reducido. En imágenes a color se posteriza cada canal por separado y luego se suman las imágenes resultantes.
- **Detección de bordes (borroneado):** pipeline de tres etapas secuencialmente dependientes entre sí:
 1. *Filtrado (blur)*: suaviza la imagen para eliminar ruido.
 2. *Resaltado (highlight)*: amplifica las diferencias entre zonas de borde y no-borde mediante aproximación de gradiente (filtros Prewitt, Sobel u otros).
 3. *Umbralado*: binariza el resultado del resaltado para obtener los bordes detectados.

La imagen cartoon final se obtiene sumando la salida de la posterización y la de la detección de bordes.

Dado el volumen de cómputo involucrado —especialmente para imágenes de grandes dimensiones—, este trabajo explora y compara distintas **estrategias de paralelización** con el fin de reducir el tiempo de procesamiento. Se implementan cuatro versiones del algoritmo:

- **Secuencial:** implementación de referencia, sin paralelismo.
- **Memoria compartida (OpenMP):** paralelismo a nivel de hilos dentro de un único nodo mediante directivas de compilador.
- **Memoria distribuida (MPI):** paralelismo mediante paso de mensajes entre procesos, potencialmente en distintos nodos.
- **Híbrido (MPI + OpenMP):** combinación de ambos paradigmas para explotar simultáneamente múltiples nodos y múltiples núcleos por nodo.

El análisis de desempeño se realiza sobre imágenes de tres tamaños (800×800 , 2000×2000 y 5000×5000 píxeles), con filtros de convolución de 3×3 y 5×5 , y post-
rizado configurado con 3 y 9 rangos de color. Las métricas utilizadas son el *speedup* y la eficiencia paralela.

Capítulo 2

Decisiones de Diseño

2.1. Paradigmas y estrategia general

Para el diseño de los algoritmos paralelos se adoptaron dos paradigmas de forma complementaria:

- **Paradigma de control** para las tareas de posterizado y borroneado: dado que ambas operan de forma independiente sobre la imagen de entrada (sin dependencias de datos entre sí), pueden ejecutarse en paralelo. Se decidió paralelizarlas asignando recursos de cómputo de forma independiente a cada una.
- **Paradigma de datos** para la distribución de los datos de entrada: dentro de cada tarea, los datos (filas o bloques de la imagen) se reparten entre los workers disponibles.

Dentro de la sección de borroneado, las dos subtareas (blur y highlight+umbralado) sí presentan dependencia secuencial entre sí, por lo que se implementa un **pipeline de datos**: primero se ejecuta la función de blur sobre toda la imagen y, sobre el resultado, se aplica la función de highlight junto con la operación de umbralado.

2.2. Algoritmo secuencial

En todas las versiones del algoritmo se aplicó la técnica de **padding**: se añade un borde auxiliar de píxeles con valor neutro (negro) alrededor de la matriz de entrada. Esto elimina la necesidad de verificar en cada iteración si las celdas vecinas son válidas al calcular el blur y el highlight, simplificando el código y mejorando la localidad de memoria.

2.3. Algoritmo con memoria compartida (OpenMP)

Se aplica descomposición de datos de entrada para cada tarea dentro del proceso.

La paralelización de las tareas de borroneado y posterizado se realiza con la directiva

```
#pragma omp parallel sections num_threads(2)
```

que identifica las dos secciones a ejecutar en paralelo, combinada con `#pragma omp section` para cada una de ellas.

Suma y posterizado. Se usa `#pragma omp parallel for schedule(static)`. La planificación estática se eligió porque el costo computacional de cada iteración es uniforme, lo que permite dividir el espacio de trabajo de forma equitativa sin sobrecarga de administración dinámica.

Blur y highlight. La inicialización de datos se paraleliza con `#pragma omp parallel for schedule(static)`. El procesamiento principal emplea

```
#pragma omp parallel for collapse(2) schedule(guided)
```

La cláusula `collapse(2)` fusiona los dos loops anidados en un único espacio de iteraciones. La planificación `guided` se adopta porque el trabajo por iteración puede ser variable o impredecible, y resulta más eficiente que `dynamic` en la administración del balance de carga.

2.4. Algoritmo con memoria distribuida (MPI)

El proceso de rango 0 actúa como **orquestador**: recibe la imagen original, la distribuye entre los workers y recolecta los resultados parciales. Los procesos worker se dividen en dos grupos según la sección asignada: la mitad procesa el posterizado y la otra mitad el borroneado.

Durante el análisis experimental se observó que paralelizar la **suma final** de las matrices de posterizado y borroneado incrementaba el tiempo de ejecución de forma contraproducente. Este comportamiento se atribuye a la latencia y overhead de comunicación entre procesos, que supera el costo de la suma en sí misma. Por esta razón se decidió **no paralelizar la suma final**, dejándola a cargo del proceso orquestador.

2.5. Algoritmo híbrido (MPI + OpenMP)

El algoritmo híbrido fue creado:

- MPI: Se reutilizó el código generado en el algoritmo con memoria distribuida.
- OpenMP: Se agregaron a los bucles del anterior código, las directrices que se utilizaron previamente en el algoritmo con memoria compartida.

La suma que se realiza al final para juntar los resultados del posterizado y borroneado se decidió que se paralelizara utilizando únicamente OpenMP, como se vio en el anterior algoritmo, los tiempos aumentaban al paralelizarla con MPI pero OpenMP al menos en un entorno local resulta ser mucho más eficiente.